MRV System — Architecture & Technical Specification

1. Overview

Next Generation MRV (Monitoring, Reporting, and Verification) Prototype for the World Bank's Department for Climate Change. Enables online collaboration on ISFL Monitoring Reports between World Bank (FMT), Participating Countries, and Auditor Entities.

Scope constraints (from SOW): - English-only, web-only (no mobile) - MVP limited to maximum 2 participating countries - AWS Consumer infrastructure (not GovCloud) - No 3rd party integrations - ISFL Monitoring Report template only

Note on IaC: The SOW specifies "AWS CDK and/or Serverless" for IaC. This implementation uses SST v3 (Pulumi-based), which provides higher-level serverless constructs on AWS. This deviation should be approved by the client.

2. Tech Stack

Layer	Technology	Version
Runtime	Node.js	20 LTS
Frontend	Next.js (App Router) + React + TypeScript	Next.js 14+, React 18+
UI Framework	Tailwind CSS + Catalyst (Tailwind UI premium kit)	Tailwind 3.4+
Backend	AWS Lambda (Node.js) via SST v3	SST Ion (v3)
Infrastructure as Code	SST v3 (Pulumi)	Latest
Database	Aurora Serverless v2 (PostgreSQL)	PostgreSQL 15
ORM	Drizzle ORM	Latest
Authentication	Amazon Cognito	-
Storage	Amazon S3	-
Rich Text Editor	TipTap	v2
PDF Export	@react-pdf/renderer	-
Testing	Vitest (unit/integration), Playwright (E2E)	-
Package Manager	pnpm	9+
Monorepo	pnpm workspaces	-

3. AWS Services Map

Service	Purpose
Aurora Serverless v2	PostgreSQL database (report data, users, templates)
Lambda	API function handlers (Node.js)
API Gateway (HTTP API)	REST API routing, throttling, JWT authorizer

Service	Purpose
Cognito	User authentication, user pools, groups for RBAC
S3	PDF exports, file attachments
CloudFront	CDN for Next.js frontend static assets + SSR
Route 53	DNS management, custom domain
ACM	SSL/TLS certificates for custom domains
WAF	Web Application Firewall on CloudFront + API Gateway
VPC	Network isolation for Aurora + Lambda
RDS Proxy	Connection pooling between Lambda and Aurora
Secrets Manager	Database credentials (auto-rotated)
SES	Email notifications
CloudWatch	Logging, metrics, alarms
IAM	Roles and policies (least privilege per function)

4. Project Structure

```
worldbank-poc/
├── sst.config.ts                # SST v3 configuration
├── sst-env.d.ts                 # SST type definitions
├── infra/                       # SST infrastructure definitions
│   ├── vpc.ts                   # VPC, subnets, security groups
│   ├── database.ts              # Aurora Serverless v2 + RDS Proxy
│   ├── auth.ts                  # Cognito User Pool, groups, client
│   ├── storage.ts               # S3 buckets
│   ├── api.ts                   # API Gateway + Lambda functions
│   ├── email.ts                 # SES configuration
│   ├── monitoring.ts            # CloudWatch dashboards, alarms
│   └── web.ts                   # Next.js deployment (CloudFront)
├── packages/
│   ├── core/                    # Business logic & domain models
│   │   ├── src/
│   │   │   ├── domain/          # Entity types, enums, validation (zod)
│   │   │   ├── services/        # Business logic
│   │   │   ├── auth/            # RBAC helpers, permission checks
│   │   │   └── utils/           # Shared utilities
│   │   └── package.json
│   ├── db/                      # Database layer
│   │   ├── src/
│   │   │   ├── schema/          # Drizzle schema definitions
│   │   │   ├── migrations/      # SQL migrations
│   │   │   ├── seed.ts          # Seed data for dev/test
│   │   │   └── index.ts         # DB client & connection
│   │   └── package.json
│   ├── functions/               # Lambda function handlers
│   │   ├── src/
│   │   │   ├── middleware/      # Auth, validation, error handling middleware
│   │   │   ├── auth/            # Auth callback handlers
│   │   │   ├── templates/       # Template CRUD handlers
│   │   │   ├── reports/         # Report CRUD handlers
│   │   │   ├── comments/        # Comment handlers
│   │   │   ├── organizations/   # Org & user management handlers
│   │   │   ├── notifications/   # Notification handlers
│   │   │   └── exports/         # PDF export handlers
│   │   └── package.json
```

```
│   └── web/                      # Next.js frontend
│       ├── src/
│       │   ├── app/              # App Router pages & layouts
│       │   ├── components/       # React components (Catalyst-based)
│       │   ├── hooks/            # Custom React hooks
│       │   ├── lib/              # API client, auth helpers, utils
│       │   └── styles/           # Tailwind config & globals
│       └── package.json
├── docs/                         # Documentation
├── .github/                      # GitHub Actions CI/CD
│   └── workflows/
│       └── deploy.yml
├── pnpm-workspace.yaml
├── package.json
└── tsconfig.json
```

---

5. SST v3 Infrastructure Configuration

5.1 sst.config.ts Skeleton

```
/// <reference path="./.sst/platform/config.d.ts" />
export default $config({
  app(input) {
    return {
      name: "worldbank-mrv",
      removal: input?.stage === "prod" ? "retain" : "remove",
      home: "aws",
      providers: { aws: { region: "us-east-1" } },
    };
  },
  async run() {
    // Infrastructure modules imported in dependency order
    const vpc = await import("./infra/vpc");
    const database = await import("./infra/database");
    const auth = await import("./infra/auth");
    const storage = await import("./infra/storage");
    const email = await import("./infra/email");
    const api = await import("./infra/api");
    const web = await import("./infra/web");
    const monitoring = await import("./infra/monitoring");
  },
});
```

5.2 Key SST Constructs

Resource	SST Construct	Notes
Database	`new sst.aws.Postgres()`	Aurora Serverless v2, linked to VPC
Auth	`new sst.aws.CognitoUserPool()`	With groups: fmt, country, auditor
API	`new sst.aws.ApiGatewayV2()`	HTTP API with JWT authorizer
Frontend	`new sst.aws.Nextjs()`	Linked to API and Auth
Storage	`new sst.aws.Bucket()`	For PDFs and attachments

Resource	SST Construct	Notes
Email	`new sst.aws.Email()`	SES for notifications

5.3 Resource Linking

SST v3 uses `link` to inject resource references into Lambda functions:

```
// infra/api.ts
const api = new sst.aws.ApiGatewayV2("Api");
api.route("GET /templates", {
  handler: "packages/functions/src/templates/list.handler",
  link: [database.cluster, auth.userPool, storage.bucket],
});
```

Functions access linked resources via `Resource`:

```
// packages/functions/src/templates/list.ts
import { Resource } from "sst";
const dbUrl = Resource.Database.url; // injected by SST link
```

---

6. Network Architecture (VPC)

6.1 VPC Layout

```
VPC CIDR: 10.0.0.0/16

├── Public Subnets (2 AZs)
│   ├── 10.0.1.0/24 (us-east-1a) — NAT Gateway
│   └── 10.0.2.0/24 (us-east-1b) — NAT Gateway (HA)
│
├── Private Subnets — Application (2 AZs)
│   ├── 10.0.10.0/24 (us-east-1a) — Lambda ENIs
│   └── 10.0.11.0/24 (us-east-1b) — Lambda ENIs
│
└── Private Subnets — Database (2 AZs)
    ├── 10.0.20.0/24 (us-east-1a) — Aurora, RDS Proxy
    └── 10.0.21.0/24 (us-east-1b) — Aurora, RDS Proxy
```

6.2 Security Groups

Security Group	Inbound	Outbound
`sg-lambda`	None	All (for NAT/VPC endpoints)
`sg-rds-proxy`	TCP 5432 from `sg-lambda`	TCP 5432 to `sg-aurora`
`sg-aurora`	TCP 5432 from `sg-rds-proxy`	None

6.3 VPC Endpoints (to avoid NAT for AWS services)

- **S3**: Gateway endpoint (free)
- **Secrets Manager**: Interface endpoint
- **CloudWatch Logs**: Interface endpoint
- **SES**: Interface endpoint (or use NAT)

7. User Roles & RBAC

7.1 Role Definitions

Role	Cognito Group	Description
FMT Admin	`fmt`	Full system admin — manage templates, orgs, users, review reports
Country User	`country`	Fill and submit reports, handle revisions
Auditor	`auditor`	Review assigned reports, mark complete

7.2 Authorization Matrix

Endpoint	FMT	Country	Auditor
Templates			
`POST /templates`	Yes	No	No
`PUT /templates/:id`	Yes (own drafts)	No	No
`POST /templates/:id/publish`	Yes	No	No
`GET /templates`	All	Published only	Published only
`DELETE /templates/:id`	Yes (drafts only)	No	No
Reports			
`POST /reports`	No	Yes	No
`GET /reports`	All	Own country only	Assigned countries only
`GET /reports/:id`	Yes	Own country only	Assigned country only
`PUT /reports/:id/sections/:sectionId`	No	Own, DRAFT status only	No
`DELETE /reports/:id`	No	Own, DRAFT only	No
`POST /reports/:id/submit`	No	Own, DRAFT only	No
`POST /reports/:id/request-revision`	During FMT review	No	During auditor review
`POST /reports/:id/forward-to-auditor`	Yes (FMT approved)	No	No
`POST /reports/:id/complete`	No	No	Assigned only
Comments			
`POST /reports/:id/comments`	During review	Reply only (during revision)	During review
`POST /comments/:id/resolve`	Yes	Own comments	Yes
Organizations			
All org/user endpoints	Yes	No	No
Notifications			

Endpoint	FMT	Country	Auditor
`GET /notifications`	Own	Own	Own

7.3 Data Isolation Rules

- **Country users** can only access reports belonging to their `organization_id`. Every query for country users MUST include `WHERE country_id = (user's country)`.
- **Auditor users** can only access reports for countries where their organization is the assigned auditor (`countries.auditor_organization_id = user.organization_id`).
- **FMT users** have access to all data (admin scope).
- Data isolation is enforced at the **service layer** in `packages/core/src/services/`. Every service function receives the authenticated user context and applies scoping filters.

---

8. Authentication Flow

8.1 Cognito Configuration

Setting	Value
Auth flow	USER_SRP_AUTH (Secure Remote Password)
Username	Email address
Required attributes	email, given_name, family_name
Custom attributes	custom:organization_id
Password policy	Min 8 chars, uppercase, lowercase, number, symbol
MFA	Optional (TOTP) — not enforced in MVP
Access token expiry	1 hour
Refresh token expiry	30 days
ID token expiry	1 hour

8.2 User Pool Groups

Three Cognito groups created during infrastructure setup: `fmt`, `country`, `auditor`. Users are assigned to groups when invited by FMT admin.

8.3 Frontend Auth Flow

1. User enters email + password on login page
2. Frontend uses AWS Amplify Auth library (`signIn()`) with SRP
3. Cognito returns ID token, access token, refresh token
4. Tokens stored in memory (not localStorage) + httpOnly cookie for refresh
5. API client attaches `Authorization: Bearer <accessToken>` to all requests
6. Token refresh happens automatically via Amplify when access token expires
7. On 401 from API, redirect to login

8.4 API Gateway JWT Authorizer

API Gateway HTTP API uses a built-in JWT authorizer: - Issuer: Cognito User Pool URL - Audience: Cognito App Client ID - Extracts `sub`, `cognito:groups`, `custom:organization_id` from token claims - Lambda handlers receive user

context in `event.requestContext.authorizer.jwt.claims`

---

9. Database Schema

9.1 Enums

```
CREATE TYPE org_type AS ENUM ('WORLD_BANK', 'COUNTRY', 'AUDITOR');
CREATE TYPE user_status AS ENUM ('ACTIVE', 'INACTIVE');
CREATE TYPE user_role AS ENUM ('ADMIN', 'MEMBER');
CREATE TYPE template_status AS ENUM ('DRAFT', 'PUBLISHED', 'ARCHIVED');
CREATE TYPE editor_type AS ENUM ('RICH_TEXT', 'TABLE');
CREATE TYPE report_status AS ENUM (
  'DRAFT',
  'SUBMITTED_TO_FMT',
  'FMT_REVIEW_IN_PROGRESS',
  'FMT_REVISION_REQUESTED',
  'FMT_APPROVED',
  'SUBMITTED_TO_AUDITOR',
  'AUDITOR_REVIEW_IN_PROGRESS',
  'AUDITOR_REVISION_REQUESTED',
  'COMPLETED'
);
CREATE TYPE comment_status AS ENUM ('OPEN', 'RESOLVED');
CREATE TYPE notification_type AS ENUM (
  'REPORT_SUBMITTED',
  'REVISION_REQUESTED',
  'REPORT_RESUBMITTED',
  'REPORT_COMPLETED',
  'NEW_TEMPLATE_VERSION'
);
```

9.2 Tables

organizations

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
name	varchar(255)	NOT NULL
type	org_type	NOT NULL
status	user_status	NOT NULL, default ‘ACTIVE’
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

users

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
cognito_sub	varchar(255)	UNIQUE, NOT NULL
email	varchar(255)	UNIQUE, NOT NULL
first_name	varchar(255)	NOT NULL

Column	Type	Constraints
last_name	varchar(255)	NOT NULL
organization_id	uuid	FK -> organizations.id, NOT NULL
role	user_role	NOT NULL, default 'MEMBER'
status	user_status	NOT NULL, default 'ACTIVE'
last_login_at	timestamptz	NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Indexes: `users(cognito_sub)`, `users(email)`, `users(organization_id)`

countries

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
name	varchar(255)	NOT NULL
iso_code	varchar(3)	UNIQUE, NOT NULL
organization_id	uuid	FK -> organizations.id, UNIQUE, NOT NULL
auditor_organization_id	uuid	FK -> organizations.id, NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Indexes: `countries(organization_id)`, `countries(auditor_organization_id)`

report_templates

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
name	varchar(255)	NOT NULL
description	text	NULL
version	integer	NOT NULL, default 1
status	template_status	NOT NULL, default 'DRAFT'
template_group_id	uuid	NOT NULL (links all versions of same template)
created_by	uuid	FK -> users.id, NOT NULL
published_at	timestamptz	NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Versioning strategy: Each version is a new row. All versions of the same logical template share the same `template_group_id`. The latest published version is found via `WHERE template_group_id = X AND status = 'PUBLISHED' ORDER BY version DESC LIMIT 1`.

Indexes: `report_templates(template_group_id, status, version)`, `report_templates(status)`

template_sections

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
template_id	uuid	FK -> report_templates.id, ON DELETE CASCADE, NOT NULL
title	varchar(255)	NOT NULL
instructions	jsonb	NOT NULL, default '{}' (TipTap JSON)
editor_type	editor_type	NOT NULL, default 'RICH_TEXT'
editor_config	jsonb	NOT NULL, default '{}'
sample_data	jsonb	NULL (pre-filled content for user)
sort_order	integer	NOT NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

`editor_config` shape for TABLE type:

```
{
  "columns": [
    { "key": "year", "label": "Year", "type": "number", "required": true },
    { "key": "value", "label": "Emission Value", "type": "number", "required": true },
    { "key": "source", "label": "Data Source", "type": "text", "required": false }
  ]
}
```

Indexes: `template_sections(template_id, sort_order)`

reports

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
template_id	uuid	FK -> report_templates.id, NOT NULL
template_version	integer	NOT NULL
country_id	uuid	FK -> countries.id, NOT NULL
status	report_status	NOT NULL, default 'DRAFT'
current_revision	integer	NOT NULL, default 0
created_by	uuid	FK -> users.id, NOT NULL
submitted_at	timestamptz	NULL
completed_at	timestamptz	NULL
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Indexes: `reports(country_id, status)`, `reports(status)`, `reports(template_id)`

report_sections

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
report_id	uuid	FK -> reports.id, ON DELETE CASCADE, NOT NULL
template_section_id	uuid	FK -> template_sections.id, NOT NULL
content	jsonb	NOT NULL, default '{}' (TipTap JSON or table data)
is_complete	boolean	NOT NULL, default false
last_edited_by	uuid	FK -> users.id, NULL
last_edited_at	timestamptz	NULL
version	integer	NOT NULL, default 1 (for optimistic locking)
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Indexes: `report_sections(report_id)`

report_revisions

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
report_id	uuid	FK -> reports.id, ON DELETE CASCADE, NOT NULL
revision_number	integer	NOT NULL
content_snapshot	jsonb	NOT NULL (full report content at time of submission)
submitted_by	uuid	FK -> users.id, NOT NULL
submitted_to	varchar(20)	NOT NULL ('fmt' or 'auditor')
submitted_at	timestamptz	NOT NULL, default now()
created_at	timestamptz	NOT NULL, default now()

Indexes: `report_revisions(report_id, revision_number)`

comments

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
report_id	uuid	FK -> reports.id, ON DELETE CASCADE, NOT NULL
section_id	uuid	FK -> report_sections.id, NOT NULL
revision_id	uuid	FK -> report_revisions.id, NOT NULL
parent_comment_id	uuid	FK -> comments.id, NULL (for threads)
author_id	uuid	FK -> users.id, NOT NULL
content	text	NOT NULL
status	comment_status	NOT NULL, default 'OPEN'
resolved_by	uuid	FK -> users.id, NULL
resolved_at	timestamptz	NULL

Column	Type	Constraints
created_at	timestamptz	NOT NULL, default now()
updated_at	timestamptz	NOT NULL, default now()

Indexes: `comments(report_id, section_id)`, `comments(report_id, status)`

change_records

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
report_id	uuid	FK -> reports.id, ON DELETE CASCADE, NOT NULL
section_id	uuid	FK -> report_sections.id, NOT NULL
from_revision_id	uuid	FK -> report_revisions.id, NOT NULL
to_revision_id	uuid	FK -> report_revisions.id, NOT NULL
previous_content	jsonb	NOT NULL
new_content	jsonb	NOT NULL
created_at	timestamptz	NOT NULL, default now()

Indexes: `change_records(report_id, section_id)`

notifications

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
user_id	uuid	FK -> users.id, ON DELETE CASCADE, NOT NULL
type	notification_type	NOT NULL
title	varchar(255)	NOT NULL
message	text	NOT NULL
link	varchar(500)	NULL (deep link to relevant page)
is_read	boolean	NOT NULL, default false
created_at	timestamptz	NOT NULL, default now()

Indexes: `notifications(user_id, is_read, created_at DESC)`

audit_log

Column	Type	Constraints
id	uuid	PK, default gen_random_uuid()
user_id	uuid	FK -> users.id, NOT NULL
action	varchar(100)	NOT NULL (e.g., ‘report.submit’, ‘comment.create’)
entity_type	varchar(50)	NOT NULL (e.g., ‘report’, ‘template’)
entity_id	uuid	NOT NULL
metadata	jsonb	NULL (additional context)

Column	Type	Constraints
created_at	timestamptz	NOT NULL, default now()

Indexes: `audit_log(entity_type, entity_id)`, `audit_log(user_id, created_at DESC)`

---

10. Report Workflow State Machine

10.1 States

State	Description	Editable by Country?
`DRAFT`	Report being filled by country user	Yes
`SUBMITTED_TO_FMT`	Awaiting FMT review	No
`FMT_REVIEW_IN_PROGRESS`	FMT actively reviewing	No
`FMT_REVISION_REQUESTED`	FMT returned with comments	Yes
`FMT_APPROVED`	FMT approved, ready for auditor	No
`SUBMITTED_TO_AUDITOR`	Forwarded to assigned auditor	No
`AUDITOR_REVIEW_IN_PROGRESS`	Auditor actively reviewing	No
`AUDITOR_REVISION_REQUESTED`	Auditor returned with comments	Yes
`COMPLETED`	Auditor marked as complete (terminal)	No

10.2 Valid Transitions

```
DRAFT ──────────────────────► SUBMITTED_TO_FMT
SUBMITTED_TO_FMT ──────────► FMT_REVIEW_IN_PROGRESS
FMT_REVIEW_IN_PROGRESS ────► FMT_REVISION_REQUESTED
FMT_REVIEW_IN_PROGRESS ────► FMT_APPROVED
FMT_REVISION_REQUESTED ────► DRAFT (country re-edits)
FMT_APPROVED ──────────────► SUBMITTED_TO_AUDITOR
SUBMITTED_TO_AUDITOR ──────► AUDITOR_REVIEW_IN_PROGRESS
AUDITOR_REVIEW_IN_PROGRESS ► AUDITOR_REVISION_REQUESTED
AUDITOR_REVIEW_IN_PROGRESS ► COMPLETED
AUDITOR_REVISION_REQUESTED ► DRAFT (country re-edits)
```

10.3 Transition Rules

Transition	Triggered by	Guard conditions
DRAFT → SUBMITTED_TO_FMT	Country user submits	All sections marked complete
SUBMITTED → FMT_REVIEW_IN_PROGRESS	FMT user opens report	Auto-transition on first access
FMT_REVIEW → FMT_REVISION_REQUESTED	FMT clicks “Request Revision”	At least 1 open comment
FMT_REVIEW → FMT_APPROVED	FMT clicks “Approve”	No open comments
FMT_REVISION_REQUESTED → DRAFT	Country begins editing	Auto-transition on edit

Transition	Triggered by	Guard conditions
FMT_APPROVED → SUBMITTED_TO_AUDITOR	FMT clicks “Forward to Auditor”	Country has assigned auditor
SUBMITTED → AUDITOR_REVIEW_IN_PROGRESS	Auditor opens report	Auto-transition on first access
AUDITOR_REVIEW → AUDITOR_REVISION_REQUESTED	Auditor clicks “Request Revision”	At least 1 open comment
AUDITOR_REVIEW → COMPLETED	Auditor clicks “Mark Complete”	No open comments
AUDITOR_REVISION_REQUESTED → DRAFT	Country begins editing	Auto-transition on edit

The state machine is implemented in `packages/core/src/services/report-workflow.ts` with explicit validation of allowed transitions.

---

11. API Specification

11.1 Common Conventions

Base URL: `https://api.{domain}/`

Authentication: `Authorization: Bearer <cognito_access_token>`

Standard Response Envelope:

```
{
  "data": { ... },
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 42,
    "nextCursor": "abc123"
  }
}
```

Error Response:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human readable message",
    "details": [
      { "field": "name", "message": "Required" }
    ]
  }
}
```

HTTP Status Codes: | Code | Usage | |——|——-| | 200 | Success (GET, PUT) | | 201 | Created (POST) | | 204 | No content (DELETE) | | 400 | Validation error | | 401 | Unauthenticated | | 403 | Forbidden (wrong role / not authorized) | | 404 | Resource not found | | 409 | Conflict (optimistic locking, duplicate) | | 500 | Internal server error |

Pagination: Cursor-based for list endpoints. Query params: `?limit=20&cursor=<opaque_string>`. Default limit: 20, max: 100.

Input Validation: All request bodies validated with Zod schemas in `packages/core/src/domain/`. Validation runs in Lambda middleware before handler logic.

11.2 Auth Endpoints

```
POST   /auth/callback          # Cognito post-auth callback (if using hosted UI)
GET    /auth/me                # Get current user profile
                                Response: { data: User }
POST   /auth/change-password   # Change own password
                                Body: { oldPassword, newPassword }
```

Note: Login/signup/forgot-password are handled directly by Cognito SDK on the frontend (not via API).

11.3 Template Endpoints

```
GET    /templates              # List templates (FMT: all, others: published only)
                                Query: ?status=PUBLISHED&limit=20&cursor=x
                                Response: { data: Template[], meta }

POST   /templates              # Create new template [FMT only]
                                Body: { name, description }
                                Response: { data: Template } (201)

GET    /templates/:id          # Get template with sections
                                Response: { data: Template & { sections: TemplateSection[] } }

PUT    /templates/:id          # Update template metadata [FMT, DRAFT only]
                                Body: { name?, description? }
                                Response: { data: Template }

DELETE /templates/:id          # Delete draft template [FMT, DRAFT only]
                                Response: 204

POST   /templates/:id/publish  # Publish template [FMT only]
                                Response: { data: Template }

POST   /templates/:id/clone    # Clone as new draft version [FMT only]
                                Response: { data: Template } (201)

POST   /templates/:id/sections # Add section [FMT, DRAFT only]
                                Body: { title, instructions, editor_type, editor_config?, sample_data
                                Response: { data: TemplateSection } (201)

PUT    /templates/:id/sections/:sectionId  # Update section
                                Body: { title?, instructions?, editor_type?, editor_config?, sample_d

DELETE /templates/:id/sections/:sectionId  # Delete section
                                Response: 204

PUT    /templates/:id/sections/reorder     # Batch reorder sections
                                Body: { order: [{ id: string, sort_order: number }] }
```

11.4 Report Endpoints

```
GET    /reports                # List reports (scoped by role)
                                Query: ?status=DRAFT&country_id=x&limit=20&cursor=x
                                Response: { data: Report[], meta }

POST   /reports                # Create report from template [Country only]
                                Body: { template_id }
                                Response: { data: Report } (201)
```

```
GET    /reports/:id             # Get report with sections
                                 Response: { data: Report & { sections: ReportSection[] } }

DELETE /reports/:id             # Delete draft report [Country, DRAFT only]
                                 Response: 204

PUT    /reports/:id/sections/:sectionId    # Save section content [Country, DRAFT only]
                                 Body: { content: JSON, is_complete: boolean }
                                 Headers: If-Match: <version> (optimistic locking)
                                 Response: { data: ReportSection }
                                 409 if version conflict

POST   /reports/:id/validate   # Check completeness [Country]
                                 Response: { data: { complete: boolean, missing: string[] } }

POST   /reports/:id/submit     # Submit to FMT [Country, DRAFT only]
                                 Response: { data: Report }

POST   /reports/:id/request-revision   # Request revision [FMT or Auditor, during review]
                                 Body: { message?: string }
                                 Response: { data: Report }

POST   /reports/:id/approve    # Approve report [FMT, during FMT review]
                                 Response: { data: Report }

POST   /reports/:id/forward-to-auditor  # Forward to auditor [FMT, FMT_APPROVED]
                                 Response: { data: Report }

POST   /reports/:id/complete   # Mark complete [Auditor, during auditor review]
                                 Response: { data: Report }

GET    /reports/:id/revisions  # List revisions
                                 Response: { data: ReportRevision[] }

GET    /reports/:id/revisions/:revisionId  # Get specific revision content
                                 Response: { data: ReportRevision }

GET    /reports/:id/revisions/:revisionId/diff  # Diff from previous revision
                                 Response: { data: ChangeRecord[] }

GET    /reports/:id/export/pdf # Export as PDF [Any authorized user]
                                 Response: { data: { url: string } } (pre-signed S3 URL)
```

11.5 Comment Endpoints

```
GET    /reports/:id/comments   # List comments for report
                                 Query: ?section_id=x&status=OPEN&limit=50&cursor=x
                                 Response: { data: Comment[], meta }

POST   /reports/:id/comments   # Add comment [Reviewer during review, or reply]
                                 Body: { section_id, content, parent_comment_id? }
                                 Response: { data: Comment } (201)

PUT    /comments/:id           # Edit own comment
                                 Body: { content }
                                 Response: { data: Comment }

POST   /comments/:id/resolve   # Resolve comment
                                 Response: { data: Comment }

POST   /comments/:id/reopen    # Reopen resolved comment
                                 Response: { data: Comment }
```

11.6 Organization & User Endpoints

```
GET    /organizations           # List organizations [FMT only]
                                 Query: ?type=COUNTRY&limit=20&cursor=x
                                 Response: { data: Organization[], meta }

POST   /organizations           # Create organization [FMT only]
                                 Body: { name, type }
                                 Response: { data: Organization } (201)

PUT    /organizations/:id       # Update organization [FMT only]
                                 Body: { name?, status? }
                                 Response: { data: Organization }

GET    /organizations/:id/users # List users in org [FMT only]
                                 Response: { data: User[] }

POST   /organizations/:id/users # Invite user [FMT only]
                                 Body: { email, first_name, last_name, role }
                                 Response: { data: User } (201)
                                 Side effect: creates Cognito user, sends invite email

PUT    /users/:id               # Update user [FMT or self]
                                 Body: { first_name?, last_name?, role?, status? }
                                 Response: { data: User }

GET    /countries               # List countries [FMT only]
                                 Response: { data: Country[] }

POST   /countries               # Create country [FMT only]
                                 Body: { name, iso_code, organization_id }
                                 Response: { data: Country } (201)

PUT    /countries/:id           # Update country [FMT only]
                                 Body: { name? }
                                 Response: { data: Country }

POST   /countries/:id/assign-auditor  # Assign auditor to country [FMT only]
                                 Body: { auditor_organization_id }
                                 Response: { data: Country }
```

11.7 Notification Endpoints

```
GET    /notifications           # List own notifications
                                 Query: ?is_read=false&limit=20&cursor=x
                                 Response: { data: Notification[], meta }

PUT    /notifications/:id/read  # Mark as read
                                 Response: 204

POST   /notifications/read-all  # Mark all as read
                                 Response: 204

GET    /notifications/unread-count  # Get unread count (for badge)
                                 Response: { data: { count: number } }
```

---

12. Frontend Architecture

12.1 App Router Route Structure

```
app/
├── (auth)/                     # Auth layout (no sidebar)
```

```
│   ├── login/page.tsx
│   ├── forgot-password/page.tsx
│   └── layout.tsx
├── (dashboard)/                # Main app layout (Catalyst SidebarLayout)
│   ├── layout.tsx              # SidebarLayout + Header + NotificationBell
│   ├── page.tsx                # Dashboard home
│   ├── admin/
│   │   ├── organizations/
│   │   │   ├── page.tsx        # Org list (Catalyst Table)
│   │   │   └── [id]/page.tsx   # Org detail + user list
│   │   └── countries/
│   │       └── page.tsx        # Country list + auditor assignment
│   ├── templates/
│   │   ├── page.tsx            # Template list
│   │   ├── new/page.tsx        # Create template
│   │   ├── [id]/
│   │   │   ├── edit/page.tsx   # Template builder
│   │   │   └── preview/page.tsx
│   ├── reports/
│   │   ├── page.tsx            # Report list (filtered by role)
│   │   ├── new/page.tsx        # Create report (select template)
│   │   ├── [id]/
│   │   │   ├── edit/page.tsx   # Report editor (section nav + editor)
│   │   │   ├── review/page.tsx # Review interface (FMT/Auditor)
│   │   │   └── revisions/page.tsx
│   └── notifications/
│       └── page.tsx            # Full notification list
└── layout.tsx                  # Root layout (providers, fonts)
```

12.2 Catalyst Component Mapping

UI Pattern	Catalyst Component
App shell	`SidebarLayout`
Data tables	`Table`, `TableHead`, `TableBody`, `TableRow`, `TableCell`
Forms	`Field`, `Label`, `Input`, `Select`, `Textarea`
Dialogs	`Dialog`, `DialogTitle`, `DialogBody`, `DialogActions`
Buttons	`Button` (primary, secondary, outline variants)
Badges	`Badge` (for status indicators)
Navigation	`Sidebar`, `SidebarItem`, `SidebarSection`
Dropdowns	`Dropdown`, `DropdownButton`, `DropdownMenu`, `DropdownItem`
Descriptions	`DescriptionList`, `DescriptionTerm`, `DescriptionDetails`

12.3 State Management

- **Server data:** React Query (TanStack Query) for API data fetching, caching, and mutations. Stale-while-revalidate pattern.
- **Auth state:** React context provider wrapping Amplify auth.
- **Editor state:** TipTap manages its own state; synced to server via debounced save.
- **UI state:** Local React state (useState/useReducer). No global state library needed.

12.4 Auto-Save Strategy

For report section editing: 1. TipTap `onUpdate` fires on every content change 2. Debounce 2 seconds after last keystroke 3. Send `PUT /reports/:id/sections/:sectionId` with `If-Match: <version>` header 4. On 409 (version

conflict), show “This section was modified. Reload to see changes.” 5. On success, update local version number 6. Visual indicator: “Saving…” → “Saved” → auto-hide after 2s

---

13. Infrastructure Configuration

13.1 Aurora Serverless v2

Setting	Dev	Production
Min ACU	0.5	2
Max ACU	2	16
PostgreSQL version	15	15
Multi-AZ	No	Yes (automatic)
Backup retention	7 days	30 days
Point-in-time recovery	Yes	Yes
Deletion protection	No	Yes
Credentials	Secrets Manager (auto-rotated)	Secrets Manager (auto-rotated)

13.2 Lambda Configuration

Function Group	Memory	Timeout	Notes
API handlers (CRUD)	256 MB	15s	Standard API operations
PDF export	1024 MB	60s	react-pdf rendering needs more memory
Auth handlers	256 MB	10s	Cognito triggers
DB migrations	512 MB	120s	Run via SST console or CLI

Cold start mitigation: SST v3 configures Lambda functions with optimal bundling (esbuild, tree-shaking). For production, consider provisioned concurrency on critical API handlers if cold starts become an issue.

13.3 API Gateway (HTTP API)

Setting	Value
Type	HTTP API (v2) — cheaper than REST API
Default throttle	1000 requests/second (burst: 2000)
Request payload limit	10 MB
Custom domain	api.{domain} via Route 53 + ACM
CORS	Allow origin: frontend domain, methods: GET/POST/PUT/DELETE, headers: Authorization/Content-Type

13.4 S3 Configuration

Setting	Value
Bucket naming	`{stage}-worldbank-mrv-storage`
Key structure	`reports/{reportId}/revisions/{revisionId}/export.pdf`
Access	Block all public access; pre-signed URLs for downloads (1h expiry)
Encryption	SSE-S3 (default encryption)
Versioning	Enabled
Lifecycle	Move to IA after 90 days, Glacier after 365 days

13.5 WAF Rules

Applied to CloudFront distribution and API Gateway: - Rate limiting: 2000 requests per 5 minutes per IP - AWS Managed Rules: Core rule set (CRS), SQL injection, XSS - Geographic restriction: None (World Bank operates globally)

13.6 CloudWatch Monitoring

- **Lambda**: Error rate alarm (>1% over 5min), duration P99 alarm (>5s)
- **Aurora**: CPU >80% alarm, connections >80% of max alarm
- **API Gateway**: 5xx rate alarm (>1% over 5min), latency P99 alarm
- **Structured logging**: JSON format from all Lambda functions
- **Log retention**: 30 days (dev), 90 days (prod)

---

14. Deployment Architecture

14.1 Environments

Stage	AWS Account	Purpose
`dev`	Development	Local development with SST dev mode
`staging`	Staging	User acceptance testing, pre-prod
`prod`	Production	Live system

SST stages map to separate CloudFormation stacks within the same or different accounts.

14.2 CI/CD Pipeline

GitHub Actions workflow: 1. **On PR:** Lint, type-check, unit tests, integration tests 2. **On merge to main:** Auto-deploy to `staging` 3. **On release tag:** Deploy to `prod` (manual approval gate)

14.3 Database Migrations

- Migrations run as a separate Lambda function triggered after deployment
- `packages/db/src/migrations/` contains ordered SQL migration files
- Drizzle Kit generates migrations from schema changes
- Migration Lambda is invoked by SST's `afterDeploy` hook

---

15. Local Development Setup

15.1 Prerequisites

- Node.js 20 LTS
- pnpm 9+
- AWS CLI configured with appropriate profile
- Docker (optional, for local PostgreSQL)

15.2 Quick Start

```
# 1. Clone and install
git clone <repo>
cd worldbank-poc
pnpm install

# 2. Start SST dev mode (provisions cloud resources)
npx sst dev

# 3. In another terminal, start Next.js dev server
pnpm --filter web dev

# 4. Seed the database (creates FMT org + admin user)
pnpm --filter db run seed

# 5. Open http://localhost:3000
```

15.3 Environment Variables

SST injects all resource values automatically via `Resource` bindings. No `.env` file needed for linked resources. For local overrides:

```
# .env.local (web package only, for Next.js)
NEXT_PUBLIC_API_URL=<from SST dev output>
NEXT_PUBLIC_COGNITO_USER_POOL_ID=<from SST dev output>
NEXT_PUBLIC_COGNITO_CLIENT_ID=<from SST dev output>
NEXT_PUBLIC_AWS_REGION=us-east-1
```